HARE: HumAn pRiors, a key to small language model Efficiency

Lingyun Zhang*, **Bin jin***, **Gaojian Ge**, **Lunhui Liu**,
Xuewen Shen, **Mingyong Wu**, **Houqian Zhang**, **Yongneng Jiang**, **Shiqi Chen**, **Shi Pu***†
LiteAI China Telecom Guizhou Branch
{zhangly41, jinb1, gegj, liulh5, shenxw, wumy16, zhanghq20, jiangyn4, chensq27, pus1}@chinatelecom.cn

arXiv:2406.11410v2 [cs.CL] 18 Jun 2024

Abstract

Human priors play a crucial role in efficiently utilizing data in deep learning. However, with the development of large language models (LLMs), there is an increasing emphasis on scaling both model size and data volume, which often diminishes the importance of human priors in data construction. Influenced by these trends, existing Small Language Models (SLMs) mainly rely on web-scraped large-scale training data, neglecting the proper incorporation of human priors. This oversight limits the training efficiency of language models in resource-constrained settings. In this paper, we propose a principle to leverage human priors for data construction. This principle emphasizes achieving high-performance SLMs by training on a concise dataset that accommodates both semantic diversity and data quality consistency, while avoiding benchmark data leakage. Following this principle, we train an SLM named HARE-1.1B [1]. Extensive experiments on large-scale benchmark datasets demonstrate that HARE-1.1B performs favorably against state-of-the-art SLMs, validating the effectiveness of the proposed principle. Additionally, this provides new insights into efficient language model training in resource-constrained environments from the view of human priors.

1 Introduction

Small language models (SLMs) have recently garnered significant attention for their computational efficiency and real-time responsiveness (Mehta et al., 2024; Zhang et al., 2024; Li et al., 2023; Singer et al., 2024). However, influenced by the scaling law of large language models (LLMs), existing SLMs (Mehta et al., 2024; Zhang et al., 2024) rely heavily on web-scraped large-scale data. This reliance limits their efficient training in resource-constrained settings due to inconsistent quality and a lack of semantic diversity in the data. With limited parameters, SLMs face significant training challenges compared to LLMs (Radford et al., 2019). To mitigate these issues, (Li et al., 2023) and (Ben Allal et al., 2024) synthesize textbook-quality training data using high-performance LLMs, achieving competitive performance with less training data. The success of (Li et al., 2023) and (Ben Allal et al., 2024) can be attributed to incorporating human priors in data construction. These methods cluster large-scale web-scraped data into multiple topics and use diverse prompts along with topic-specific data as seed data for data synthesis. This process enhances semantic diversity, reflecting the human prior that diversity is crucial for improving model generalization. Additionally, using LLMs to filter and govern web-scraped data ensures consistent data quality. This aligns with human priors on the need for data consistency to avoid the negative impacts of noise and errors on model training. This motivation to incorporate human priors parallels the approach of effective supervised learning models (Xia et al., 2021; Von Rueden et al., 2021), where increasing sample diversity and applying consis-

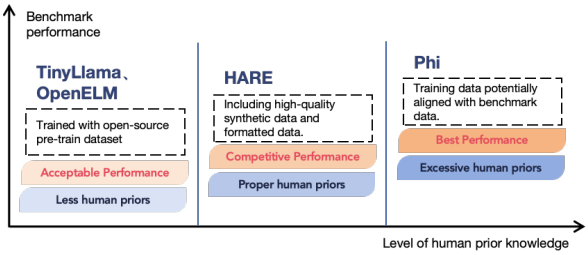


Figure 1: Illustration showing the performances of representative SLMs (Zhang et al., 2024; Mehta et al., 2024; Li et al., 2023) on the benchmark datasets in the open LLM leaderboard (Beeching et al., 2023) at different levels of human prior knowledge. As the integration of human priors deepens, models demonstrate improved performance, but with an increased risk of data leakage.

*Equal contribution.
†Corresponding author.
[1] https://github.com/LiteAI-Team/HARE

tent labels enhance the quality of the training data. By embedding these human priors into the training data, models trained under better conditions, thus enhancing performance and generalization. The incorporation of human priors helps (Li et al., 2023) achieve outstanding performance on benchmark datasets. However, we observe that some recent SLMs (Bai et al., 2023; Li et al., 2023; Bellagente et al., 2024) may inject excessive human priors, risking benchmark data leakage as shown in Figure 1 and Table 2. Based on our analysis of human priors in existing works, we propose a data construction principle: ensuring both semantic diversity and data quality consistency while avoiding benchmark data leakage. We noted that (Ben Allal et al., 2024) is the most similar to our proposed principle. However, this approach does not achieve satisfactory performance on benchmark evaluations. This is because, while injecting semantic diversity, it does not incorporate Natural Language Processing (NLP) task semantics in natural language form, resulting in a lack of NLP-task solving capability on benchmarks. (Radford et al., 2019) and (Brown et al., 2020) demonstrate that LLMs can learn zero-shot capabilities for NLP tasks from vast amounts of data. For SLMs to acquire the same capability from limited data, they must rely on the injection of human priors.

Therefore, guided by this principle, we propose our data construction method: (1) Extract high-quality, clearly categorized data from large-scale web-scraped datasets to ensure semantic diversity and maintain consistently high data quality. (2) Cluster large-scale web-scraped data into various topics and use topic-specific data with diverse prompts to generate synthetic data using high-performance LLMs, which enhance semantic diversity while ensuring consistent data quality. (3) Construct large-scale NLP-task data in natural language form to enhance semantic diversity and improve NLP-task solving capabilities. (4) Implement strict decontamination procedures to avoid benchmark data leakage. Using this method, we constructed a training dataset and trained an SLM named HARE-1.1B. This model performs favorably against existing state-of-the-art (SOTA) SLMs on large-scale benchmark datasets, validating our proposed principle and data construction method. We hope this work will inspire further exploration in incorporating human priors through the design of network architecture, loss functions, and regularization to enhance SLM training efficiency, not just through efficient data utilization.

2 Data Construction

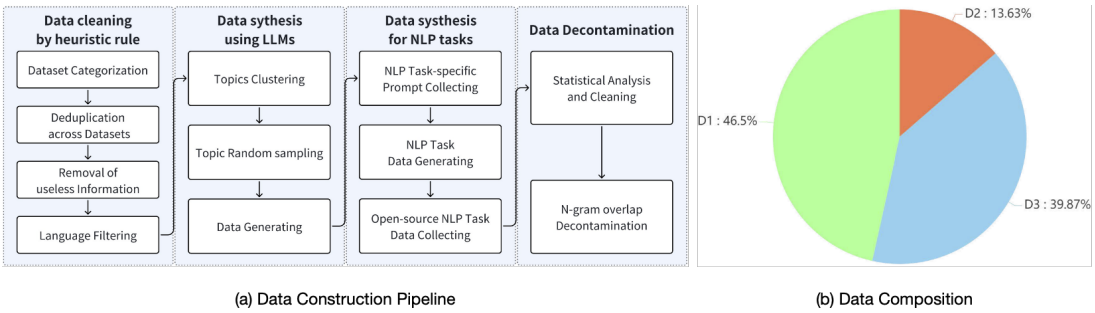
In this section, we detail our data construction pipeline, as illustrated in Figure 2(a).

Data cleaning by heuristic rule. We implement a series of heuristic rules to clean collected open-source pre-training corpora, including RedPajama (Computer, 2023), Pile (Gao et al., 2020), OpenWebMath (Paster et al., 2023), and others. Appendix A.1 provides a comprehensive list of our collected data. We categorize these datasets according to their sources and adjust proper sampling weights to enhance semantic diversity. The heuristic rules we apply include deduplication across datasets, removal of personal privacy information and web links, elimination of semantically incomplete samples, and exclusion of non-English samples. These processes ensure that the cleaned data consistently maintain high quality. We designate the cleaned data as D_1.

Data synthesis using LLMs. Even after heuristic rule-based cleaning, unresolved semantic ambiguities, such as missing text paragraphs or formatting errors, may still exist. Therefore, we utilize the Mixtral-8×7B (Jiang et al., 2024) model for data synthesis based on a subset extracted from D_1. Appendix A.2 provides details of this subset. First, we cluster the subset data into various topics and sample seed data from these topics. We then input diverse prompts coupled with the seed data into the Mixtral-8×7B model for data synthesis. Appendix A.2 presents the details of the data synthesis process. This approach significantly enhances semantic diversity through diverse topics and prompts, while using the same LLM for data synthesis ensures consistent data quality. We set the synthesized data as D_2.

Data synthesis for NLP tasks. To enhance NLP-task solving capabilities, we construct a substantial amount of NLP-task data in natural language form. Specifically, we create numerous prompts and use a subset of D_2 as seed data to guide the synthesis of diverse NLP-task data using Qwen1.5-32B (Bai et al., 2023). Additionally, we collect various open-source NLP-task datasets to further expand this dataset. Appendix A.3 provides details of the data synthesis process. We refer to this dataset for NLP tasks as D_3.

Data decontamination. To ensure that our generated data poses no risk of benchmark data leak-



(a) Data Construction Pipeline (b) Data Composition

Figure 2: Data Construction Pipeline and Composition. D_1: high-quality categorized open-source cleaned data. D_2: semantic enriched synthetic data. D_3: mixture of synthetic and open-source NLP task data.

age, we establish a rigorous data decontamination process. First, we conduct statistical analyses to remove samples that did not meet our standards, such as those with excessively short or long context lengths. Subsequently, we calculate the n-gram overlap with benchmark data to eliminate potentially duplicated samples. Appendix A.4 details the decontamination process.

Our final training dataset consists of D_1, D_2, and D_3. Figure 2(b) shows the sample weights of the three parts.

3 Training

We use the Mistral (Jiang et al., 2023) architecture and reduce the model parameters to approximately 1.1B. The modified model consists of 22 hidden layers, 32 attention heads, and 8 key-value heads, with a hidden size of 2048 and a vocabulary size of 32,000. We name the pre-trained model HARE-1.1B. HARE-1.1B is trained on 16 Nvidia-H800 GPUs using DeepSpeed (Rasley et al., 2020) and Flash-Attention (Dao et al., 2022). The maximum sequence length is set to 2048, with a batch size of 2 million tokens and a maximum learning rate of 3e-4. The training process is divided into two stages based on different data sources. In the first stage, we train our model on the D_1 dataset, processing 52 billion tokens. In the second stage, HARE-1.1B is trained on our final training dataset, which includes D_1, D_2, and D_3. The entire training process spans 30 days, during which 600 billion tokens are processed. Note that the number of processed tokens is far smaller than that of (Zhang et al., 2024) (3T tokens), (Mehta et al., 2024) (1.8T tokens), and (Singer et al., 2024) (2T tokens). Additionally, we have fine-tuned the trained 1.1B model for chat and android api calling applications, details can be found in the Appendix C.

4 Experiments

In this section, we conduct ablation studies and comparisons with SOTA SLMs, including Phi1.5 (Li et al., 2023), Qwen1.5 (Bai et al., 2023), Stablelm2 (Bellagente et al., 2024), H2o-danube (Singer et al., 2024), openELM (Mehta et al., 2024), Csg-wukong (OpenCSG, 2024), Cosmo (Ben Allal et al., 2024), TinyLlama (Zhang et al., 2024), and Gpt2xl (Radford et al., 2019). These studies are performed on the Open LLM Leaderboard (Beeching et al., 2023) using the MMLU (Hendrycks et al., 2021), ARC-C (Clark et al., 2018), TruthfulQA (Lin et al., 2022), Winogrande (Sakaguchi et al., 2019), Hellaswag (Zellers et al., 2019), and Gsm8k (Cobbe et al., 2021) benchmark datasets. We follow the standard benchmark protocols on the Open LLM Leaderboard.

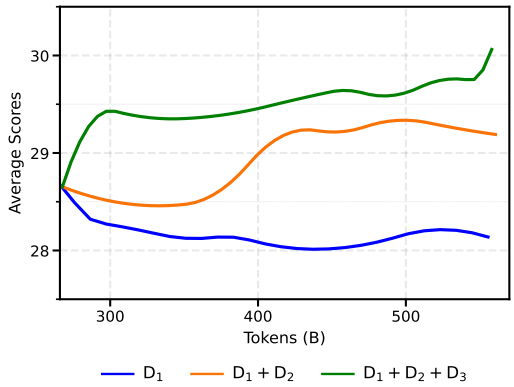


Figure 3: Ablation studies showing the average score curves of 0.25B models on the Open LLM Leaderboard as the number of training tokens increases.

4.1 Ablation studies

Due to limited computing resources, we conduct ablation studies on a 0.25B model to efficiently validate the effectiveness of our proposed data construction method. We establish three experimental groups: (1) training on the D_1 dataset; (2) training on the combination of the D_1 and D_2 datasets; and (3) training on the integration of the D_1, D_2, and

Model	Size	Average	MMLU	ARC-C	TruthfulQA	Winogrande	Hellaswag	Gsm8k
Phi1.5	1.3B	47.69	43.89	52.9	40.89	72.22	63.79	12.43
Qwen1.5	1.8B	46.55	46.71	37.88	39.43	60.3	61.42	33.59
Stablelm2	1.6B	45.25	38.95	43.34	36.78	64.56	70.45	17.44
HARE	1.1B	40.17	35.74	38.4	42.08	59.27	57.46	8.04
H2o-danube	1.8B	39.12	25.94	39.42	33.86	64.48	69.58	1.44
OpenELM	1.1B	38.47	27.05	36.69	33.86	63.22	65.71	1.21
Csg-wukong	1B	37.78	25.33	37.71	42.79	56.67	58.93	5.23
Cosmo	1B	36.59	26.69	38.57	38.15	55.49	55.13	5.53
TinyLlama	1.1B	36.42	26.04	33.87	37.32	59.51	60.31	1.44
Gpt2xl	1.6B	34.31	26.55	30.29	38.53	58.01	51.39	1.06

Table 1: Comparisons with the SOTA SLMs on Open LLM leaderboard. HARE performs favorably against existing SLMs in terms of average score.

Model	Gsm8k		ARC		Hellaswag		MMLU		Winogrande		TruthfulQA	
	Δ_{train}	Δ_{test}	Δ_{train}	Δ_{test}	Δ_{train}	Δ_{test}	Δ_{train}	Δ_{test}	Δ_{train}	Δ_{test}	Δ_{train}	Δ_{test}
Phi1.5	1.7	0.62	21.8	-0.15	0.54	-	-	-13.87	7.65	-	9.18	-
Qwen1.5	35.01	10.94	0.51	0.32	0.94	-	-	1.85	-0.13	-	6.9	-
Stablelm2	9.26	0.28	1.56	0.85	2.15	-	-	-0.23	-0.15	-	3.85	-
HARE	7.63	4.35	-0.52	-0.25	1.31	-	-	-2.61	-0.16	-	4.77	-
H2o-danube	0.29	0.25	0.01	0	0.03	-	-	-0.5	-0.02	-	1.15	-
OpenELM	0.7	0.9	0.54	0.25	1.42	-	-	-0.44	-0.25	-	9.14	-
Csg-wukong	11.27	9.05	0.37	-0.22	0.3	-	-	-0.25	-0.15	-	4.33	-
Cosmo	0.24	-0.71	-0.11	-0.03	0.62	-	-	0	-0.22	-	4.48	-
TinyLlama	1.16	1.03	-0.29	-0.05	1.21	-	-	0.11	-0.32	-	5.67	-
Gpt2xl	0.74	0.76	0.55	0.62	0.49	-	-	-0.33	-0.25	-	4.78	-

Table 2: Evaluation of benchmark data leakage using the method from (Xu et al., 2024). The greater the value of Δ, the higher the probability of benchmark data leakage. Red: highest score, Blue: lowest score. HARE maintains relatively low levels of Δ across comparison, indicating a lower probability of data leakage.

D_3 datasets. The three groups use the same parameter weights to initialize the model and the same hyperparameters during training, differing only in the composition of training data. The objective is to explore how different data combinations affect model performance. Figure 3 shows a gradual improvement in model performance with the addition of D_2 and D_3. This is because the addition of D_2 enhances the semantic diversity and consistency of data quality in the training dataset, while the inclusion of D_3 supplements the dataset with task-specific data for NLP tasks. Consequently, this improves the performance of our model on the benchmark datasets. The results of the ablation studies support using our final dataset, including D_1, D_2, and D_3, to train our HARE-1.1B model.

4.2 Overall performance

We compare our HARE-1.1B with 9 SLMs on the Open LLM Leaderboard and evaluate these models for benchmark data leakage using the method from (Xu et al., 2024). Appendix B presents the details of this method. Table 1 shows that HARE performs well on the Open LLM Leaderboard. Specifically, in terms of average score, our model outperforms models trained with web-scraped large-scale data, including (Zhang et al., 2024; Mehta et al., 2024; Singer et al., 2024), and models trained on synthetic data generated by LLMs lacking NLP-task data (Ben Allal et al., 2024). These results demonstrate the effectiveness of our data construction method, which incorporates human priors in training data and enables our model to achieve favorable performance. Although our model does not perform as well as the top 3 SLMs, Phi1.5, Qwen1.5, and Stablelm2, in terms of average score, we note that Table 2 shows these models have significant Δ values on several benchmark train and test dataset evaluations, indicating a substantial risk of benchmark data leakage. In contrast, our model maintains relatively low Δ values across all benchmark evaluations. The results from Table 1 and Table 2 effectively validate that our data construction method enhances model capabilities without introducing benchmark data leakage issues.

5 Conclusion

In this paper, we propose encoding human priors into data construction for training SLMs. Our data construction method ensures both semantic diversity and data quality consistency, while avoiding benchmark data leakage. We conducted extensive

evaluations on various benchmarks to validate the effectiveness of the proposed method. The HARE-1.1B model, trained on the dataset built using this method, performs favorably against SOTA SLMs.

Limitations

Our work explores the application of human priors in training SLMs, which entails certain limitations. Despite recognizing the importance of human priors, we lack a discussion on the quality of these priors. Inappropriately selected or biased human priors could lead to suboptimal or even misleading model outcomes, which will be explored and addressed in future work. Furthermore, due to constraints in computational resources, we were unable to fully explore the constraints between parameters of SLMs and human prior knowledge, leaving the qualitative relationship undetermined.

Acknowledgments

Thanks to everyone who has contributed to this paper, including data collection and synthesis, model training, comparative experiments, and manuscript drafting. We are particularly thankful to Qun Zeng, who initially guided the direction of our work and provided meticulous guidance throughout. We also extend our appreciation to Shan Ma, Manjia Ding, and Daiyong Huang for their substantial support and help in our endeavors. The authors also wish to acknowledge the support of China Telecom Guizhou Branch for providing unwavering support.

References

Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609*.

Edward Beeching, Clémentine Fourrier, Nathan Habib, Sheon Han, Nathan Lambert, Nazneen Rajani, Omar Sanseviero, Lewis Tunstall, and Thomas Wolf. 2023. Open llm leaderboard.

Marco Bellagente, Jonathan Tow, Dakota Mahan, Duy Phung, Maksym Zhuravinskyi, Reshinth Adithyan, James Baicoianu, Ben Brooks, Nathan Cooper, Ashish Datta, et al. 2024. Stable lm 2 1.6 b technical report. *arXiv preprint arXiv:2402.17834*.

Loubna Ben Allal, Anton Lozhkov, Guilherme Penedo, Thomas Wolf, and Leandro von Werra. 2024. Cosmopedia.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *NeurIPS*.

Wei Chen and Zhiyuan Li. 2024. Octopus v2: On-device language model for super agent. *Preprint*, arXiv:2404.01744.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *Preprint*, arXiv:1803.05457.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *Preprint*, arXiv:2110.14168.

Together Computer. 2023. Redpajama: an open dataset for training large language models.

Mike Conover, Matt Hayes, Ankit Mathur, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin. 2023. Free dolly: Introducing the world's first truly open instruction-tuned llm.

Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *NeurIPS*.

Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. 2023. Enhancing chat language models by scaling high-quality instructional conversations. *Preprint*, arXiv:2305.14233.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. 2020. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring massive multitask language understanding. *Preprint*, arXiv:2009.03300.

Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7b. *arXiv preprint arXiv:2310.06825*.

Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088*.

Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, and Yin Tat Lee. 2023. Textbooks are all you need ii: phi-1.5 technical report. *Preprint*, arXiv:2309.05463.

Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. Truthfulqa: Measuring how models mimic human falsehoods. *Preprint*, arXiv:2109.07958.

Sachin Mehta, Mohammad Hossein Sekhavat, Qingqing Cao, Maxwell Horton, Yanzi Jin, Chenfan Sun, Iman Mirzadeh, Mahyar Najibi, Dmitry Belenko, Peter Zatloukal, and Mohammad Rastegari. 2024. Openelm: An efficient language model family with open training and inference framework. *Preprint*, arXiv:2404.14619.

Moonshot.AI. 2023. Kimi.

OpenCSG. 2024. Csg-wukong.

Keiran Paster, Marco Dos Santos, Zhangir Azerbayev, and Jimmy Ba. 2023. Openwebmath: An open dataset of high-quality mathematical web text. *arXiv preprint arXiv:2310.06786*.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog*.

Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *SIGKDD*.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2019. WINOGRANDE: an adversarial winograd schema challenge at scale. *Preprint*, arXiv:1907.10641.

Philipp Singer, Pascal Pfeiffer, Yauhen Babakhin, Maximilian Jeblick, Nischay Dhankhar, Gabor Fodor, and Sri Satish Ambati. 2024. H2o-danube-1.8b technical report. *Preprint*, arXiv:2401.16818.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. Llama: Open and efficient foundation language models. *Preprint*, arXiv:2302.13971.

Laura Von Rueden, Sebastian Mayer, Katharina Beckh, Bogdan Georgiev, Sven Giesselbach, Raoul Heese, Birgit Kirsch, Julius Pfrommer, Annika Pick, Rajkumar Ramamurthy, et al. 2021. Informed machine learning–a taxonomy and survey of integrating prior knowledge into learning systems. *IEEE TKDE*.

Tingyu Xia, Yue Wang, Yuan Tian, and Yi Chang. 2021. Using prior knowledge to guide bert's attention in semantic textual matching tasks. In *WWW*.

Ruijie Xu, Zengzhi Wang, Run-Ze Fan, and Pengfei Liu. 2024. Benchmarking benchmark leakage in large language models. *arXiv preprint arXiv:2404.18824*.

Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. 2023. Metamath: Bootstrap your own mathematical questions for large language models. *arXiv preprint arXiv:2309.12284*.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? *Preprint*, arXiv:1905.07830.

Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. Tinyllama: An open-source small language model. *Preprint*, arXiv:2401.02385.

Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. 2023. Automatic chain of thought prompting in large language models. In *ICLR*.

A Data Construction

A.1 Collected open-source data

Table 3 shows our collected open-source pre-training corpora. We clean these collected data to create D_1, with a sample weight of 46.5%.

Source	Categories	Sample weights(%)
RedPajamaC4	C4	11%
RedPajamaArxiv	Arxiv	1.42%
Pubmed	References	0.65%
S2orc	Open Research Corpus	1.68%
PhiPapers	Philosophy	0.06%
RedPajamaBook	Book	1.73%
PG_19	Book	1.46%
RedPajamaStackExchange	Net	1.10%
HackerNews	News	0.10%
FreeLaw	Law	0.46%
PileofLaw	Law	0.34%
AMPS	Khan Academy	0.96%
DM_math	Math	0.5%
Orca_math	Math	2.56%
OpenWebMath	Math	3.88%
Fanfics	Book	1.32%
RedPajamaWiki	Wiki	7.78%
The-Stack	Code	5.5%
StackOverFlow	Code	4%

Table 3: **Open-source Pre-training Corpora Overview**: The open-source pre-training corpora are categorized.

A.2 Data synthesis using LLMs

We sample seeds from the sources in D_1 shown in Table 4. Specific prompts are designed for different data sources, and Mixtral-8×7B is used to generate synthetic data according to these prompts and seeds. We generate approximately 28 billion synthesized tokens. An example prompt used for generation is shown in this section. We combine our synthetic data with the cosmopedia dataset as D_2. The sample weights of our synthetic data and the cosmopedia dataset are 7.7% and 5.93%, respectively.

Seed source	Categories	Synthetic tokens(B)
RedPajamaC4	Web	4.5
RedPajamaWiki	Wiki	8.16
HackerNews	News	4.5
RedPajamaArxiv	Academic	4.62
FanFics	Books	4.08
Pile-of-law	Law	2.16
The-Stack,StackOverFlow	Code	0.06
Open-Web-Math	Math	0.12

Table 4: **Open-source Datasets for Data Synthesis**: Approximately 28B tokens for D_2 are synthesized based on D_1.

Prompt for Law Data Generation

Write a long and very detailed law course unit for a textbook, The course should be inspired from this text snippet:\n"{}"\n while trying to be: - Rigorous - you create challenging textbooks that cover the material in depth. \n - Engaging - your textbooks have a narrative arc and engaging tone, like the writing of Michael Lewis. \n - Applied - you use specific and practical examples. For example, if the topic is integration in calculus, include equations and proofs of the concept you're teaching. \n As another example, if the topic is the history of the United States, include dates, names, and key events. Do not include a title or an introduction, simply write the content without headlines and introductory phrases. Focus purely on the subject itself without offering advice on teaching and instruction. The word count of the textbook needs to be greater than 1000 words.

A.3 Mixture of synthetic and open-source NLP task data

A subset of D_2 is sampled for synthesising NLP task data. We select various NLP tasks, including Question-Answering (Q&A), Multi-Choice Q&A, Cloze, Summarization and more. Multiple prompts are designed for each task, and Qwen-32B-Chat is used for data generation. About 8 billion tokens are synthesized, and the synthetic NLP task data are referred to as "Restruct" in Table 5. An example prompt used for NLP task generation can be found in this secion. We also collect open-source SFT data and make slight format modifications, converting it into various NLP task formats. This, combined with synthetic data, constitutes the final D_3 dataset. The sample weights of the Restruct dataset and the open-source SFT dataset are 23.5% and 16.37%, respectively.

Data	Source	Tokens(B)
Restruct	$D_2\backslash Wiki$, $D_2\backslash Code$, $D_2\backslash Math$, $D_2\backslash Books$, $D_2\backslash Law$	8
Open-source SFT	OpenHermes2.5, Auto-cot, Dolly and etc.	10.97

Table 5: **Mixture of Synthetic and Open-source NLP Task Data Overview**: Approximately 18.97B tokens of NLP task data are generated for D_3.

Prompt for Multi-Choice Q&A Data Generation

Please play the role of a data generator. Your task is to generate a data sample in the feild of {topic}.
Task Requirements:
- You will be given a content about {topic}, you should read carefully and try to generate data with the content.
- You will be given some examples, which you should learn and try to generate data in the form of the given examples.
- Do NOT copy any of the examples given to you!!!
- You should focus on the quality of the generated data samples not quantity.
Output Format: Please output in the following format. {"question": "questions you generated", "options": ["option1", ..., "option4"], "answer": "gold option"}
Content: {content}
Examples: {examples}

A.4 Data decontamination process

All synthetic data will be rigorously decontaminated with benchmark datasets. We first examined the statistical characteristics of the synthetic data, removing samples with excessively short or long context lengths. In D_2, the text contextual length is restricted to approximately 2k tokens, while in D_3, the NLP task data is limited within 512 tokens. Subsequently, we check the N-gram repetition rate between the generated data and benchmark datasets. All samples in benchmark datasets are used for decontamination. For long context length data in D_2, we set N in range of 10 to 15, while in D_3 is between 4 to 8. Finally, We remove samples with a repetition rate exceeding 50%, to ensure that all synthetic data are at no risk of data leakage.

B Data Leakage

We employ the method described in (Xu et al., 2024) to evaluate benchmark data leakage. Specifically, we use GPT3.5-turbo to generate new datasets derived from the original benchmark datasets by modifying sentence structures or reordering answers. Subsequently, we assess the SLMs using accuracy scores for 5-grams with identical input to measure the performance differences between the original and new datasets. The evaluation scores are presented in Table 6. $D^{'}$ and D represent the performance of the models on synthetic and original datasets, respectively. Δ measures the extent of memorization of the models on the original training or test datasets. A higher Δ indicates deeper memorization of the training or test datasets, and a higher likelihood of data leakage. When Δ is less than zero, it suggests that the model generalizes better on synthetic data. Since the comparison involves models of the same parameter scale, no normalization is applied to Δ. Ideally, a model should perform similarly on both original and synthesized datasets, as reflected by Δ_{train} and Δ_{test}. For a model that does not utilize train and test datasets, there will not be significant Δ due to changes in sentence descriptions or the order of options.

In the evaluation on GSM8K, Qwen1.5 demonstrates significant Δ_{train} and Δ_{test} values, indicating a higher likelihood of data leakage. During the evaluations of the ARC, Winogrande, and TruthfulQA datasets, Phi1.5 demonstrates a much greater Δ_{train} compared to similarly scaled SLMs, indicating an increased potential for data leakage. Stablelm2 exhibits the largest Δ_{train} in the Hellaswag evaluation, indicating a higher possibility of data leakage. In the evaluation of MMLU, we assess the zero-shot capabilities of models by taking the first non-empty character from model outputs as the response. Except for Qwen1.5 and TinyLlama-3T, models tend to prefer the phrasing and options of synthetic data, resulting in lower Δ. Moreover, Phi1.5 and HARE demonstrate stronger zero-shot capabilities compared to other models. Our model maintains relatively low levels of Δ across comparisons, indicating a lower probability of data leakage, while also demonstrating strong generalization capabilities across various datasets. All datasets used in this evaluation will be made publicly available.

C Supervised Fine-tuning

C.1 Chat

We fine-tune HARE-1.1B on a dataset comprising 0.25B tokens sourced from Dolly (Conover et al., 2023), MetaMathQA (Yu et al., 2023), UltraChat200k (Ding et al., 2023), Auto-Cot (Zhang et al., 2023), and other collections. All data are structured into a chat template featuring roles labeled as system, user, and assistant. Full fine-tuning is conducted on the model using 8×A800

Task	Model	D'_{train}	D_{train}	Δ_{train}(-)	D'_{test}	D_{test}	Δ_{test}(-)
Gsm8k	Phi1.5	15.59	17.29	1.7	14.86	15.48	0.62
	Qwen1.5	31.56	66.57	35.01	28.8	39.74	10.94
	Stablelm2	18.78	28.02	9.24	18.44	19.12	0.68
	HARE	22.32	29.95	7.63	21.87	26.22	4.35
	H2o-danube	0.34	0.63	0.29	0.34	0.59	0.25
	OpenELM	10.23	10.93	0.7	10.34	11.24	0.9
	Csg-wukong	11.32	22.59	11.27	11.04	20.09	9.05
	Cosmo	17.89	18.13	0.24	17.43	16.72	-0.71
	TinyLlama	8.99	10.15	1.16	9.24	10.27	1.03
	Gpt2xl	3.77	4.51	0.74	3.74	4.5	0.76
ARC-C	Phi1.5	7.42	29.22	21.8	4.69	4.54	-0.15
	Qwen1.5	1.3	1.81	0.51	1.3	1.62	0.32
	Stablelm2	1.89	3.45	1.56	2.2	3.05	0.85
	HARE	7.76	7.24	-0.52	7.13	6.88	-0.25
	H2o-danube	0.04	0.05	0.01	0.05	0.05	0
	OpenELM	5.54	6.08	0.54	5.43	5.68	0.25
	Csg-wukong	2.92	3.29	0.37	2.92	2.7	-0.22
	Cosmo	2.79	2.68	-0.11	2.68	2.65	-0.03
	TinyLlama	5.33	5.04	-0.29	5.05	5	-0.05
	Gpt2xl	1.15	1.7	0.55	0.97	1.59	0.62
Hellaswag	Phi1.5	1.25	1.79	0.54	-	-	-
	Qwen1.5	1.19	2.13	0.94	-	-	-
	Stablelm2	1.36	3.51	2.15	-	-	-
	HARE	2.14	3.45	1.31	-	-	-
	H2o-danube	0	0.03	0.03	-	-	-
	OpenELM	2.24	3.66	1.42	-	-	-
	Csg-wukong	1.33	1.63	0.3	-	-	-
	Cosmo	1.45	2.07	0.62	-	-	-
	TinyLlama	2.05	3.26	1.21	-	-	-
	Gpt2xl	0.96	1.45	0.49	-	-	-
Winogrande	Phi1.5	11.13	18.78	7.65	-	-	-
	Qwen1.5	0.52	0.39	-0.13	-	-	-
	Stablelm2	0.54	0.39	-0.15	-	-	-
	HARE	1.03	0.87	-0.16	-	-	-
	H2o-danube	0.02	0	-0.02	-	-	-
	OpenELM	1	0.75	-0.25	-	-	-
	Csg-wukong	0.43	0.28	-0.15	-	-	-
	Cosmo	0.66	0.44	-0.22	-	-	-
	TinyLlama	0.96	0.64	-0.32	-	-	-
	Gpt2xl	0.42	0.25	-0.17	-	-	-
TruthfulQA	Phi1.5	7.32	16.50	9.18	-	-	-
	Qwen1.5	4.85	11.75	6.9	-	-	-
	Stablelm2	3.4	7.25	3.85	-	-	-
	HARE	5.12	9.89	4.77	-	-	-
	H2o-danube	0.32	1.47	1.15	-	-	-
	OpenELM	8.49	17.63	9.14	-	-	-
	Csg-wukong	5.02	9.35	4.33	-	-	-
	Cosmo	5.43	9.91	4.48	-	-	-
	TinyLlama	6.81	12.48	5.67	-	-	-
	Gpt2xl	3.4	8.18	4.78	-	-	-
MMLU	Phi1.5	-	-	-	39.47	25.67	-13.87
	Qwen1.5	-	-	-	5.27	7.12	1.85
	Stablelm2	-	-	-	1.29	1.06	-0.23
	HARE	-	-	-	10.61	8	-2.61
	H2o-danube	-	-	-	2.17	1.67	-0.5
	OpenELM	-	-	-	1	0.56	-0.44
	Csg-wukong	-	-	-	1.19	0.94	-0.25
	Cosmo	-	-	-	0	0	0
	TinyLlama	-	-	-	0.95	1.06	0.11
	Gpt2xl	-	-	-	0.33	0	-0.33

Table 6: Evaluation of benchmark data leakage using the method from (Xu et al., 2024). The greater the value of Δ, the higher the probability of benchmark data leakage. HARE maintains relatively low levels of Δ across comparison, indicating a lower probability of data leakage.

Model	Size	Average	MMLU	ARC-C	TruthfulQA	Winogrande	Hellaswag	Gsm8k
Qwen1.5	1.8B	43.99	45.87	38.74	40.62	59.67	60.02	19.03
HARE	1.1B	40.00	33.62	37.46	41.49	58.88	53.03	15.54
TinyLlama	1.1B	36.26	26.22	33.53	36.79	60.22	59.38	1.44

Table 7: **Results of Chat Models on Open LLM Leaderboard**: After SFT, HARE still maintains relatively competitive performance.

API Tasks	Descriptions of APIs	Training samples	Test samples	Accuracy(%)	
				HARE-1.1B-Tool	Llama3-8B-RAG
API-0	Call the designated phone number	690	50	91.7%	96%
API-1	Call to the designated contact person	732	50	100%	72%
API-2	Search for specified content using a browser	517	50	100%	32%
API-3	Turn on the camera	531	50	100%	90%
API-4	Query weather	514	50	100%	100%
API-5	Send an email with given content to the designated recipient	810	50	100%	100%
API-6	Meaningless instructions	469	50	95%	10%

Table 8: **Android API Calling SFT Datasets and Evaluation**: Under seven API tasks, HARE-1.1B-Tool outperforms Llama3-8B-RAG in four tasks and exhibits a 26.1% higher average accuracy.

GPUs. The learning rate is set to 1e-6, with a global batch size of 1024. Additionally, we assess the performance of the chat model on the open LLM leaderboard, with the results detailed in Table 7.

C.2 Android API calling

We use the method proposed by (Chen and Li, 2024) to enable HARE-1.1B to have the capability to call Android APIs. The composition of the API Calling datasets is delineated in Table 8. These datasets are synthesized utilizing the Llama3-8B (Touvron et al., 2023), Kimi (Moonshot.AI, 2023), and Mixtral-8×7B (Jiang et al., 2024) models. Based on these datasets, we fine-tune HARE-1.1B to obtain the model HARE-1.1B-Tool, and successfully deploy its int4 quantized version on mobile devices. The inference speed reaches 8 tokens per second on a Xiaomi Redmi K40 phone. Table 8 presents the comparative evaluation results of HARE-1.1B-Tool and Llama3-8B using RAG. The evaluation of Llama3-8B-RAG involves utilizing the RAG method to identify the most relevant function descriptions based on user queries. Subsequently, this language model uses these descriptions along with the user queries to generate the expected function call commands, thereby assessing its performance on test samples. The evaluation results for HARE-1.1B-Tool are obtained by fine-tuning on API Calling datasets and then conducting evaluations on test samples.